

MySQL Database Design Exercise

Customer & Product Database with 1:Many Relationship

Data & Analytics — MySQL Lab

Learning Objectives

Upon completing this exercise, students will be able to:

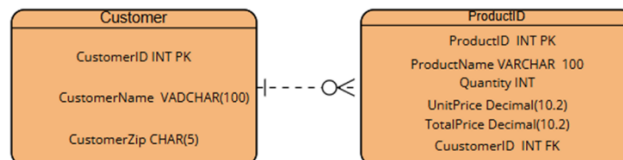
1. Design and create a relational MySQL database from scratch.
2. Define tables with appropriate fields and data types.
3. Establish a 1:Many relationship using a foreign key.
4. Insert and update records using SQL INSERT statements.
5. Verify data integrity with SELECT queries and JOINS.

Scenario

A small retail business needs a database to manage its **customers** and the **products** they have ordered. Your task is to design and build this database in MySQL, following the specifications below. Each customer may be associated with one or more product orders, establishing a **1:Many** relationship between the Customer and Product tables.

Entity-Relationship (ER) Diagram

The diagram below illustrates the two entities and their relationship. CustomerID is the primary key in the Customer table and a foreign key in the Product table, creating the 1:Many link.



CUSTOMER Table (1 side)

CUSTOMER		
Field	Data Type	Constraint
CustomerID	INT	PRIMARY KEY, AUTO_INCREMENT

CustomerName	VARCHAR(100)	NOT NULL
CustomerZip	CHAR(5)	NOT NULL

▼ One Customer → Many Products ▼

PRODUCT Table (Many side)

PRODUCT		
Field	Data Type	Constraint
ProductID	INT	PRIMARY KEY, AUTO_INCREMENT
ProductName	VARCHAR(100)	NOT NULL
Quantity	INT	NOT NULL, DEFAULT 0
UnitPrice	DECIMAL(10,2)	NOT NULL
TotalPrice	DECIMAL(10,2)	GENERATED (Qty × UnitPrice)
CustomerID	INT	FOREIGN KEY → Customer(CustomerID)

Relationship Type: 1:Many — One Customer may have many Products. CustomerID in the Product table is a FOREIGN KEY referencing Customer(CustomerID).

Table Definitions

Table 1: Customer

Attribute	Data Type	Size	Constraint	Description
CustomerID	INT	-	PK, AUTO_INCREMENT, NOT NULL	Unique customer identifier
CustomerName	VARCHAR	100	NOT NULL	Full name of the customer
CustomerZip	CHAR	5	NOT NULL	5-digit US ZIP code

Table 2: Product

Attribute	Data Type	Size	Constraint	Description
ProductID	INT	-	PK, AUTO_INCREMENT, NOT NULL	Unique product identifier
ProductName	VARCHAR	100	NOT NULL	Name of the product
Quantity	INT	-	NOT NULL, DEFAULT 0	Number of units ordered
UnitPrice	DECIMAL	10,2	NOT NULL	Price per single unit

Attribute	Data Type	Size	Constraint	Description
TotalPrice	DECIMAL	10,2	GENERATED ALWAYS	Quantity × UnitPrice (auto computed)
CustomerID	INT	-	FK → Customer(CustomerID)	Links product to its customer

Sample Data Tables

Customer Table — Initial Records

CustomerID	CustomerName	CustomerZip
1	Alice Johnson	10001
2	Bob Martinez	90210
3	Carol Thompson	60601
4	David Lee	77001
5	Eva Williams	30301

Product Table — Initial Records

ProductID	ProductName	Qty	Unit Price	Total Price	CustomerID
1	Wireless Mouse	2	\$29.99	\$59.98	1
2	USB-C Hub	1	\$49.99	\$49.99	1
3	Laptop Stand	1	\$39.95	\$39.95	2
4	Mechanical Keyboard	1	\$109.00	\$109.00	2
5	Monitor	2	\$299.99	\$599.98	3
6	Webcam HD	1	\$74.50	\$74.50	3
7	Desk Lamp	3	\$22.00	\$66.00	4
8	Noise Cancelling Headphones	1	\$199.99	\$199.99	5

Additional Records (Updates)

The following records are added to simulate new customers and product orders.

New Customer Records

CustomerID	CustomerName	CustomerZip
6	Frank Garcia	85001
7	Grace Kim	98101

New Product Records

ProductID	ProductName	Qty	Unit Price	Total Price	CustomerID
9	Smart Speaker	1	\$99.99	\$99.99	6
10	Cable Organizer	4	\$9.99	\$39.96	6
11	Drawing Tablet	1	\$249.00	\$249.00	7
12	Screen Cleaner Kit	2	\$12.50	\$25.00	7
13	Ergonomic Chair	1	\$350.00	\$350.00	1
14	Blue Light Glasses	1	\$25.00	\$25.00	3

SQL Script Overview: Algorithm

1. **Create Database** Creates StoreDB and sets it as the active database.
2. **Add Tables** — Defines the Customer and Product tables with all fields and constraints.
3. **Establish Relationship** — The FOREIGN KEY constraint on Product.CustomerID references Customer(CustomerID), creating the 1:Many relationship. ON DELETE CASCADE ensures referential integrity.
4. **Add Records** — Inserts 5 customers and 8 initial product records.
5. **Add Additional Records** — Inserts 2 more customers and 6 more product records, including new orders for existing customers.

Create a new query in the MySQL Workbench and add the following script

SQL Code Step1: Create the database

```
1      -- =====
2      -- Exercise: MySQL Database Design with Relationships
3      -- Tables: Customer and Product (1:Many Relationship)
4      -- =====
5
6      -- Deodat P. Sharma
7
8      -- =====
9      -- STEP 1: CREATE DATABASE
10     -- =====
11
12 •    CREATE DATABASE IF NOT EXISTS StoreDB;
13 •    USE StoreDB;
14
```

Output

Action Output				
#	Time	Action	Message	Duration / Fetch
✓ 1	21:06:09	CREATE DATABASE IF NOT EXISTS StoreDB	1 row(s) affected	0.031 sec
✓ 2	21:06:09	USE StoreDB	0 row(s) affected	0.000 sec

SQL Code Step 2 & 3: Add Tables and Establish Relationship

```
16  -- =====
17  -- STEP 2: ADD TABLES
18  -- =====
19
20  -- Create the Customer table (the "one" side of the relationship)
21  ● ○ CREATE TABLE Customer (
22      CustomerID  INT          NOT NULL AUTO_INCREMENT,
23      CustomerName VARCHAR(100) NOT NULL,
24      CustomerZip  CHAR(5)      NOT NULL,
25      PRIMARY KEY (CustomerID)
26  );
27
28  -- Create the Product table (the "many" side of the relationship)
29  ● ○ CREATE TABLE Product (
30      ProductID  INT          NOT NULL AUTO_INCREMENT,
31      ProductName VARCHAR(100) NOT NULL,
32      Quantity   INT          NOT NULL DEFAULT 0,
33      UnitPrice  DECIMAL(10, 2) NOT NULL,
34      TotalPrice DECIMAL(10, 2) NOT NULL,
35      CustomerID INT          NOT NULL,
36      PRIMARY KEY (ProductID),
37      -- =====
38      -- STEP 3: ESTABLISH RELATIONSHIP
39      -- Foreign key linking Product to Customer (1:Many)
40      -- One customer can have many products
41      -- =====
42      CONSTRAINT fk_customer
43          FOREIGN KEY (CustomerID)
44          REFERENCES Customer(CustomerID)
45          ON DELETE CASCADE
46          ON UPDATE CASCADE
47  );
48
```

Output

Action Output					
#	Time	Action	Message	Duration / Fetch	
✓ 1	21:08:16	CREATE TABLE Customer (CustomerID INT NOT NULL AUTO_INCRE...	0 row(s) affected	0.047 sec	
✓ 2	21:08:16	CREATE TABLE Product (ProductID INT NOT NULL AUTO_INCREM...	0 row(s) affected	0.031 sec	

SQL Code Step 4 Add Initial Records

```
50  -- =====
51  -- STEP 4: ADD INITIAL RECORDS
52  -- =====
53
54  -- Insert Customer records
55  • INSERT INTO Customer (CustomerName, CustomerZip) VALUES
56      ('Alice Johnson', '10001'),
57      ('Bob Martinez', '90210'),
58      ('Carol Thompson', '60601'),
59      ('David Lee', '77001'),
60      ('Eva Williams', '30301');
61
62  -- Insert Product records (each linked to a CustomerID)
63  -- TotalPrice = Quantity * UnitPrice (calculated manually)
64  • INSERT INTO Product (ProductName, Quantity, UnitPrice, TotalPrice, CustomerID) VALUES
65      ('Wireless Mouse', 2, 29.99, 59.98, 1),
66      ('USB-C Hub', 1, 49.99, 49.99, 1),
67      ('Laptop Stand', 1, 39.95, 39.95, 2),
68      ('Mechanical Keyboard', 1, 109.00, 109.00, 2),
69      ('Monitor', 2, 299.99, 599.98, 3),
70      ('Webcam HD', 1, 74.50, 74.50, 3),
71      ('Desk Lamp', 3, 22.00, 66.00, 4),
72      ('Noise Cancelling Headphones', 1, 199.99, 199.99, 5);
73
74
```

Output

Action Output				
#	Time	Action	Message	Duration / Fetch
✓ 1	21:10:25	INSERT INTO Customer (CustomerName, CustomerZip) VALUES (Alice Johnso...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.016 sec
✓ 2	21:10:25	INSERT INTO Product (ProductName, Quantity, UnitPrice, TotalPrice, Customerl...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.000 sec

SQL Code Step 5 Add Additional Records

```
75 -- =====
76 -- STEP 5: ADD ADDITIONAL RECORDS (Updates / New Data)
77 -- =====
78
79 -- Add two new customers
80 • INSERT INTO Customer (CustomerName, CustomerZip) VALUES
81     ('Frank Garcia', '85001'),
82     ('Grace Kim', '98101');
83
84 -- Add new products for the new customers
85 • INSERT INTO Product (ProductName, Quantity, UnitPrice, TotalPrice, CustomerID) VALUES
86     ('Smart Speaker', 1, 99.99, 99.99, 6),
87     ('Cable Organizer', 4, 9.99, 39.96, 6),
88     ('Drawing Tablet', 1, 249.00, 249.00, 7),
89     ('Screen Cleaner Kit', 2, 12.50, 25.00, 7);
90
91 -- Add additional products to existing customers
92 • INSERT INTO Product (ProductName, Quantity, UnitPrice, TotalPrice, CustomerID) VALUES
93     ('Ergonomic Chair', 1, 350.00, 350.00, 1),
94     ('Blue Light Glasses', 1, 25.00, 25.00, 3);
95
96
```

Output

Action Output				
#	Time	Action	Message	Duration / Fetch
✓ 1	21:12:48	INSERT INTO Customer (CustomerName, CustomerZip) VALUES (Frank Garcia...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 sec
✓ 2	21:12:48	INSERT INTO Product (ProductName, Quantity, UnitPrice, TotalPrice, Customerl...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.000 sec
✓ 3	21:12:48	INSERT INTO Product (ProductName, Quantity, UnitPrice, TotalPrice, Customerl...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 sec

SQL Code Verification Queries

```
97      -- =====
98      -- VERIFICATION QUERIES
99      -- =====
100
101      -- View all customers
102 •    SELECT * FROM Customer;
103
104      -- View all products with computed TotalPrice
105 •    SELECT * FROM Product;
106
107      -- View products joined with customer names
108 •    SELECT
109          c.CustomerID,
110          c.CustomerName,
111          c.CustomerZip,
112          p.ProductID,
113          p.ProductName,
114          p.Quantity,
115          p.UnitPrice,
116          p.TotalPrice
117      FROM Customer c
118      JOIN Product p ON c.CustomerID = p.CustomerID
119      ORDER BY c.CustomerID, p.ProductID;
120
```

Action Output				
#	Time	Action	Message	Duration / Fetch
✓ 1	21:14:14	SELECT * FROM Customer LIMIT 0, 1000	7 row(s) returned	0.000 sec / 0.000 sec
✓ 2	21:14:14	SELECT * FROM Product LIMIT 0, 1000	14 row(s) returned	0.000 sec / 0.000 sec
✓ 3	21:14:14	SELECT c.CustomerID, c.CustomerName, c.CustomerZip, p.ProductID,...	14 row(s) returned	0.000 sec / 0.000 sec

Result Grid   Filter Rows:			
	CustomerID	CustomerName	CustomerZip
▶	1	Alice Johnson	10001
	2	Bob Martinez	90210
	3	Carol Thompson	60601
	4	David Lee	77001
	5	Eva Williams	30301
	6	Frank Garcia	85001
	7	Grace Kim	98101
✱	NULL	NULL	NULL

Customer 1 × Product 2 Result 3

Result Grid   Filter Rows: Edit:    Export/Import: 						
	ProductID	ProductName	Quantity	UnitPrice	TotalPrice	CustomerID
▶	1	Wireless Mouse	2	29.99	59.98	1
	2	USB-C Hub	1	49.99	49.99	1
	3	Laptop Stand	1	39.95	39.95	2
	4	Mechanical Keyboard	1	109.00	109.00	2
	5	Monitor	2	299.99	599.98	3
	6	Webcam HD	1	74.50	74.50	3
	7	Desk Lamp	3	22.00	66.00	4
	8	Noise Cancelling Headphones	1	199.99	199.99	5
	9	Smart Speaker	1	99.99	99.99	6
	10	Cable Organizer	4	9.99	39.96	6
	11	Drawing Tablet	1	249.00	249.00	7
	12	Screen Cleaner Kit	2	12.50	25.00	7
	13	Ergonomic Chair	1	350.00	350.00	1
	14	Blue Light Glasses	1	25.00	25.00	3
✱	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid Filter Rows: Export: Wrap Cell Content:								
	CustomerID	CustomerName	CustomerZip	ProductID	ProductName	Quantity	UnitPrice	TotalPrice
▶	1	Alice Johnson	10001	1	Wireless Mouse	2	29.99	59.98
	1	Alice Johnson	10001	2	USB-C Hub	1	49.99	49.99
	1	Alice Johnson	10001	13	Ergonomic Chair	1	350.00	350.00
	2	Bob Martinez	90210	3	Laptop Stand	1	39.95	39.95
	2	Bob Martinez	90210	4	Mechanical Keyboard	1	109.00	109.00
	3	Carol Thompson	60601	5	Monitor	2	299.99	599.98
	3	Carol Thompson	60601	6	Webcam HD	1	74.50	74.50
	3	Carol Thompson	60601	14	Blue Light Glasses	1	25.00	25.00
	4	David Lee	77001	7	Desk Lamp	3	22.00	66.00
	5	Eva Williams	30301	8	Noise Cancelling Headphones	1	199.99	199.99
	6	Frank Garcia	85001	9	Smart Speaker	1	99.99	99.99
	6	Frank Garcia	85001	10	Cable Organizer	4	9.99	39.96
	7	Grace Kim	98101	11	Drawing Tablet	1	249.00	249.00
	7	Grace Kim	98101	12	Screen Cleaner Kit	2	12.50	25.00

Customer 1

Product 2

Result 3 ×

Discussion Questions

1. Why is CustomerID defined as AUTO_INCREMENT? What benefit does this provide?
2. What happens to Product records if a Customer is deleted? How does ON DELETE CASCADE affect this?
3. Why is TotalPrice defined as a GENERATED column instead of being manually entered?
4. How would you modify the schema if a product could belong to multiple customers?
5. Write a query that shows only customers who have more than one product ordered.